

COURSE NAME

SOFTWARE QUALITY
AND TESTING

CSC 4133

(UNDERGRADUATE)

CHAPTER 9

DOMAIN TESTING

DOMAIN TESTING

- ❑ In Domain testing or input domain testing, the overall input domain is partitioned into sub-domains and the associated boundaries, as well as the sub-domains, are tested
- ❑ The basic idea of domain testing is to generate test cases by assigning specific values to input variables based on some analyses of the input domain
 - This analysis is called domain analysis/ input domain analysis
- ❑ Basically black-box in nature, but could be white-box

DOMAIN TESTING: HOW IT WORKS

- ❑ **Identification:** Identifying the **input variable**, input space and define the input domain based on specifications (black-box) or implementation details (white-box) for the program unit under testing
- ❑ **Partitioning:** Dividing or **classifying the input domain into sub-domains** to form a partition
- ❑ **Domain Analysis:** Performing **domain analysis** for **each sub-domain to examine** its limits in each dimension and its boundary properties, such as the specific boundary definitions and the related closure properties
- ❑ **Selection:** Selecting **test points (type of error checking)** to cover these partitioned sub-domains based on domain analysis results
- ❑ **Testing:** Testing with the above selected test points as input, **checking the results (output values)**, dealing with observed problems, and carrying out analysis & follow-up activities

ERROR

- ❑ Two fundamental elements of a computer program are:
 - **Input domain:** The set of all input data to the program
 - **Program path:** A sequence of instructions from entry to exit
- ❑ A program path corresponds to some flow of control in the program.
 - **Feasible path:** A path is said to be feasible if there exists an input data which causes the program to execute the path
 - **Infeasible path:** No input data exists to cause the path to execute
- ❑ The identified two broad classes of errors by combining input domain and program paths.
 - 1) Computation error
 - 2) Domain error

COMPUTATION ERROR

- A computation error occurs when a specific input data causes the correct (desired) path to execute, but the **output value is wrong**.
- This can happen due to a wrong function being executed in an assignment statement.
- For example , Consider a desired path containing the statement

```
result = f( a, b );
```

- where a and b are input values. A computation error may occur if the statement is replaced by a faulty one, such as

```
result = f( b, a );
```

- so, the result of executing the path can be erroneous because of a fault in the assignment statement, and this can happen in spite of executing a correct path.

DOMAIN ERROR

- A domain error occurs when a specific input data causes the program to execute a **wrong path in the program**
- An incorrect path can be selected by a program if there is a fault in one or more of the conditional statements in the program
- Example: let us consider a conditional statement of the form

```
if (p) then f1()  
    else f2()
```
- If there is a **fault** in the **formulation of the predicate p** , then the wrong function call is invoked, thereby causing an incorrect path to be executed

SOURCE OF DOMAINS

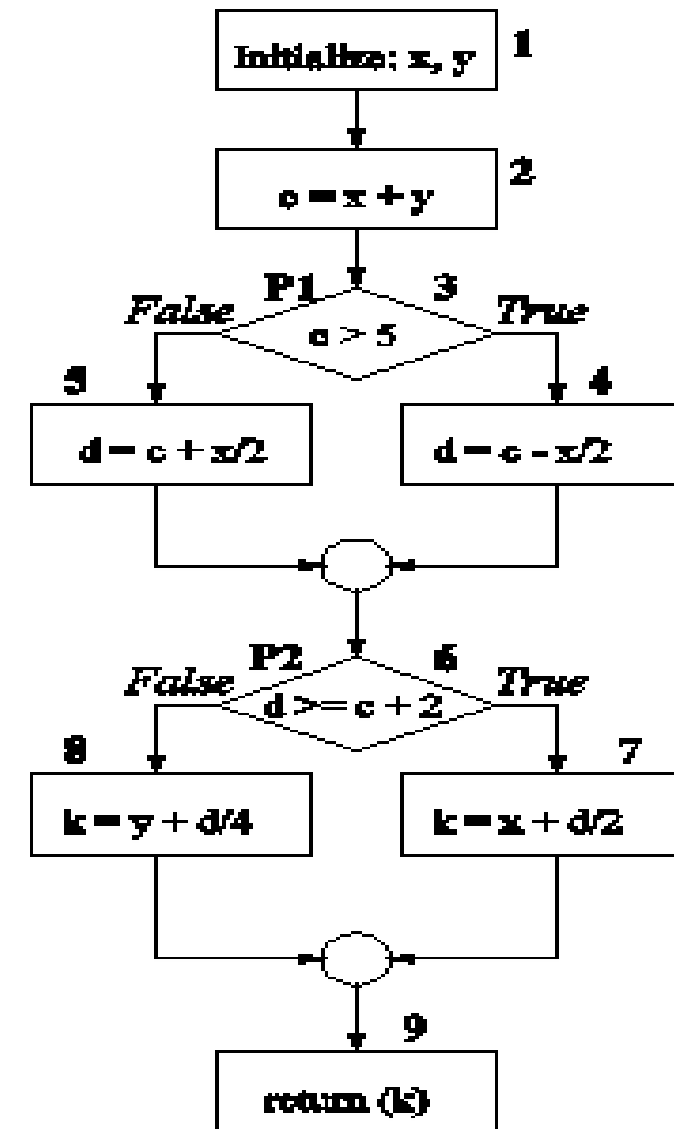
- Domains can be identified from both **specifications** and **programs**
- A method to identify domains from the source code:
 - **Draw** a control flow graph from the given **source code**
 - **Find** all possible interpretations of the **predicates**
 - **Analyze** the interpreted predicates to **identify** domains

SOURCE OF DOMAINS

```

int codedomain(int x, int y)
{  int c, d, k ;
   c = x + y;
   if (c > 5)
       d = c - x/2;
   else
       d = c + x/2;
   if (d >= c + 2)
       k = x + d/2;
   else
       k = y + d/4;
   return(k); }

```



TYPES OF DOMAIN

- ❑ A domain is defined by a set of constraints, called **boundary inequalities**
- ❑ The properties of a domain are discussed in terms of the **properties of its boundaries**
- **Closed boundary** : if the points on the boundary are included in the domain of interest ($x \leq 4$) (Here the boundary is 0 to 4)
- **Open boundary**: if the points of the boundary do not belong to the domain of interest ($x > 5$)
- **Closed domain**: if all of its boundaries are closed ($x > 2 \ \&\& \ x \leq 4$)
- **Open domain**: if some of its boundaries are open ($x > 5$)
- **Extreme point**: a point where two or more boundaries cross ($x = 0$ to 5, $y = 5$ to 10)
- **Adjacent domain**: if domains have a boundary inequality in common ($x = 0$ to 5, $y = 7$ to 10)

BOUNDARY ERROR

❑ Closure error

- Occurs if a boundary is open when the intention is to have a closed boundary, or vice versa.
- Example: the relational operator $\leq$ is implemented as $<$

❑ Shifted-boundary error

- Occurs when the implemented boundary is parallel to the intended boundary
- Example: let the intended boundary be $(x + y > 4)$, whereas the actual boundary is $(x + y > 5)$

❑ Tilted-boundary error

- Occurs if the constant coefficients of the variables in a predicate defining a boundary take up wrong values
- Example: let the intended boundary be $(x + 0.5*y > 5)$, whereas the actual boundary is $(x + y > 5)$

ON & OFF POINTS

- ❑ Data points on or near a boundary are most sensitive to find domain errors
- ❑ Based on the above idea, we define two kinds of data points near domain boundaries –
 - **ON point:** It is a point on the boundary or “very close” to the boundary
 - **OFF point:** An OFF point of a boundary lies away from the boundary
 - Example - Exam results: 0-100 marks (A= 90-100, B=80-90,)

REFERENCES

- ❑ Software Testing And Quality Assurance – Theory and Practice - Kshirasagar Naik & Priyadarshi Tripathy
- ❑ Software Quality Engineering: Testing, Quality Assurance and Quantifiable Improvement - Jeff Tian